
Chapter 02. Python Data Processing I

- 1. Pandas Object Creation (객체 생성)
- 2. Index Object (색인 객체)
- 3. Missing Data (결측치)
- 4. Selection (선택)

1. Pandas Library

- Numpy를 기반으로 만들어진 새로운 패키지, Series 와 DataFrame이라는 효율적인 자료구조 제공
- 행과 열이 단순 정수 index가 아닌 레이블로 식별되는 Numpy 배열 향상 버전

- pandas 모듈을 import 하여 사용

```
import pandas as pd
```

```
print("pd version : ", pd.__version__)
```

pandas 라이브러리 버전 확인

pd version : 1.2.5

- Pandas Series 객체
 - 인덱싱된 데이터의 1차원 배열

```
data = pd.Series([ 30, 20, -80, 75 ,73,77])  
print(data)  
print(data.values)  
print(data.index)
```

(values 만 별도로 array 확인)

(index를 이용 index 범위 확인)

[30 20 -80 75 73 77]

RangeIndex(start=0, stop=6, step=1)

0	30
1	20
2	-80
3	75
4	73
5	77
dtype: int64	

- values, index를 동시에 갖음
- list 요소 개수가 index의 개수

1.1 Series 와 DataFrame 데이터 구조

	total_bill	tip	gender	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

DataFrame : 색인을 갖는 행과 열 구조의 2차원 데이터 구조
컬럼마다 다른 데이터 형식 가능, Series가 여러 개 합쳐진 구조

Series : DataFrame 중 하나의 Column에 해당 하는 데이터 모음,
즉 색인을 갖는 1차원 배열 같은 구조

1.2 Series Object Creation (객체 생성)

1) Series 객체 생성 및 접근

```
import pandas as pd

stockinfo = pd.Series([9130.7, 9460.2, 7790.8, 7500, 7300.0, 7740.5])

print(stockinfo)

print("----value----")
print(stockinfo.values)
print(stockinfo.index)

print(stockinfo[1])

stockinfo[2] = 8540.2 # index 접근을 통한 values 값 수정

print(stockinfo[1:3])

print(stockinfo[stockinfo > 8000])
```

list 전달

```
0    9130.7
1    9460.2
2    7790.8
3    7500.0
4    7300.0
5    7740.5
dtype: float64
----value----
[9130.7 9460.2 7790.8 7500. 7300. 7740.5]
RangeIndex(start=0, stop=6, step=1)

9460.2

1    9460.2
2    8540.2
dtype: float64

0    9130.7
1    9460.2
2    8540.2
dtype: float64
```

pd.Series() 의 인자에는 [...]list / np.array배열 / 파이썬 딕셔너리를 전달 할 수 있음.

Series – numpy와 다른 특징

- 암묵적인 정수형 인덱스를 사용하는 Numpy 배열과 달리 명시적으로 정의된 인덱스 사용 가능
- 인덱스는 정수도 가능하지만 어떤 타입의 값도 가능

2) Series 객체 인덱싱 및 산술연산

```
import numpy as np
import pandas as pd
```

```
stockinfo = pd.Series([9130.7, 9460.2, 7790.8, 7500, 7300.0, 7740.5], index=["sk", "samsung", "hancom", "naver", "kakao", "kt"])
```

```
print(stockinfo)
```

```
print(stockinfo[2])
```

```
print(stockinfo['hancom'])
```

```
print(stockinfo * 2) # Series 산술연산
```

```
print("----value----")
```

```
print(stockinfo.values)
```

```
print(stockinfo.index)
```

```
sk      9130.7
samsung 9460.2
mds     7790.8
naver   7500.0
kakao   7300.0
kt      7740.5
dtype: float64
```

정수 인덱싱

라벨 인덱싱

스칼라 곱셈

----value----

[9130.7 9460.2 7790.8 7500. 7300. 7740.5]

Index(['sk', 'samsung', 'mds', 'naver', 'kakao', 'kt'], dtype='object')

```
sk      18261.4
samsung 18920.4
mds     15581.6
naver   15000.0
kakao   14600.0
kt      15481.0
dtype: float64
```

3) Series 객체 생성 및 연산 - 사전 활용

```
import numpy as np
import pandas as pd
```

```
people_kor = { '서울':500, '대전':300, '부산':200 }
obj1 = pd.Series( people_kor )
print(obj1)
```

```
local_name = [ '서울', '광주', '대구', '부산' ]
obj2 = pd.Series(obj1, index = local_name)
print(obj2)
```

```
print( obj1 + obj2 )
```

(사전의 키가 series 인덱스로 설정)

('광주', '대구' 인덱스에 대한
데이터가 없기 때문에 누락데이터
NaN 으로 처리)

(Series 간 연산시 인덱스 매칭이 되
지 않을 경우 NaN 으로 처리)

서울 500
대전 300
부산 200
dtype: int64

서울 500.0
광주 NaN
대구 NaN
부산 200.0
dtype: float64

광주 NaN
대구 NaN
대전 NaN
부산 400.0
서울 1000.0
dtype: float64

4) Series 객체 생성 - numpy array 활용

```
numdata = np.random.randint(1,10,(5,))
print(numdata)
```

```
SeriesNumData = pd.Series(numdata)
print(SeriesNumData)
```

[3 1 7 5 5] <class 'numpy.ndarray'>
0 3
1 1
2 7
3 5
4 5
5 5
dtype: int32

5) Series 객체 실습 문제

실습문제) 사전 객체 활용한 Series 객체를 생성하고 생성된 Series 객체에 'total' 인덱스를 추가하여 4개 도시 인구수의 총합을 데이터 추가

```
import pandas as pd  
  
dicData = { 'Pusan':35000, 'Daejun':71000, 'Kangju':16000, 'Seoul':980000 }  
  
# 실습 코드 추가
```

Pusan	35000
Daejun	71000
Kangju	16000
Seoul	980000
total	1102000

dtype: int64

실습문제) numpy random.randint 메서드 활용해 -10 ~ 10 사이의 5개 정수 데이터 생성, 생성된 nparray 객체를 이용한 Series 객체 생성
생성된 Series 객체 중 value가 0 보다 큰 Series 객체 생성

결과물

nparr : [2 -2 3 -10 -9]



a	2
b	-2
c	3
d	-10
e	-9

dtype: int32



a	2
c	3

dtype: int32

1.3 DataFrame Object Creation (객체 생성)

DataFrame

- 여러 개의 Series 가 합쳐진 2차원 배열 구조, 엑셀 시트와 같이 로우, 컬럼 index 구조
- 각 컬럼은 다른 타입의 자료를 가질 수 있음

1) DataFrame 객체 생성 - 사전 활용

```
import numpy as np
import pandas as pd
```

```
dict_data = { 'Name' : ['kim','park','Lee'],
              'Charge' : [5000,6000,7000],
              'local' : ['seoul','puan','jeju'],
              'CardNum' : ['9440','4805','4602'] }
```

```
df = pd.DataFrame(dict_data)
print(df)
```

```
df.index = ['one','two','three']
print(df)
```

(사전의 키는 컬럼 레이블로 사용)

	Name	Charge	local	CardNum
0	kim	5000	seoul	9440
1	park	6000	puan	4805
2	Lee	7000	jeju	4602

(색인 미 지정 시 자동 대입)

	customer	pay	local	card
one	kim	5000	seoul	9440
two	park	6000	puan	4805
three	Lee	7000	jeju	4602

* DataFrame 객체 정보확인_info() 메서드

```
import numpy as np
import pandas as pd
```

```
dict_data = { 'Name' : ['kim','park','Lee'],
              'Charge' : [5000,6000,7000],
              'local' : ['seoul','puan','jeju'],
              'CardNum' : ['9440','4805','4602'] }
```

```
df = pd.DataFrame(dict_data)
df.index = ['one','two','three']
print(df)
print(df.info()) # DataFrame 정보를 출력하고 None을 반환
```

	customer	pay	local	card
one	kim	5000	seoul	9440
two	park	6000	puan	4805
three	Lee	7000	jeju	4602

```
<class 'pandas.core.frame.DataFrame'>
Index: 3 entries, one to three
Data columns (total 4 columns):
#   Column    Non-Null Count  Dtype
---  -
0   Name      3 non-null      object
1   Charge    3 non-null      int64
2   local     3 non-null      object
3   CardNum   3 non-null      object
dtypes: int64(1), object(3)
memory usage: 120.0+ bytes
```

Process finished with exit code 0

- 해당 컬럼의 모든 데이터가 정수일 경우
Dtype : **int64**
- 해당 컬럼의 데이터가 문자와 정수 섞여
있을 경우 Dtype : **object**

2) DataFrame 객체 생성 - numpy 배열 활용

```
import numpy as np
import pandas as pd
mydf = pd.DataFrame( np.random.randint(10,size=(3,3)) ,
columns=['one','two','three'], index = ['A','B','C'] )
```

```
print(mydf)
```

column 과 index 전체 변경

```
#mydf.columns=['num1','num2','num3']
```

```
#mydf.index = [1,2,3]
```

rename 메서드 활용 column 과 index 부분 변경

```
mydf.rename( columns = {'one':'test'} , inplace = True)
```

```
mydf.rename(index={'A':'K'}, inplace = True)
```

```
print(mydf)
```

색인 객체 values 활용하여 부분 변경

```
mydf.columns.values[0] = 'fir'
```

```
mydf.index.values[0] = 't'
```

10미만의 정수 중 3행 3열을 갖는
numpy 배열 생성

객체 생성 시 컬럼 과 로우 index
지정

	one	two	three
A	5	9	2
B	5	9	7
C	6	3	7

	num1	num2	num3
1	6	5	3
2	1	9	7
3	8	6	3

	test	two	three
k	1	0	6
B	5	1	3
C	5	1	1

원본객체 바로 수정

3) DataFrame 객체 생성 – 기존 DataFrame 객체 활용

```
import numpy as np
import pandas as pd
```

```
dict_data = { 'Name' : ['kim','park','Lee'],
              'Charge' : [5000,6000,7000],
              'local' : ['seoul','pusan','jeju'],
              'CardNum' : ['9440','4805','4602'] }
```

```
mydf = pd.DataFrame( dict_data, columns = [ 'Name','deposit','CardNum','bank'] )
print(mydf)
```

(사전에 없는 컬럼 추가 시 결측치(NaN) 으로 저장)

	Name	deposit	CardNum	bank
0	kim	NaN	9440	NaN
1	park	NaN	4805	NaN
2	Lee	NaN	4602	NaN

```
# 객체 생성 후 컬럼 추가 방법
mydf['age'] = np.NaN
print(mydf)
```

	Name	deposit	CardNum	bank	age
0	kim	NaN	9440	NaN	NaN
1	park	NaN	4805	NaN	NaN
2	Lee	NaN	4602	NaN	NaN

1.4 DataFrame 객체생성 실습 문제

실습문제) 결과물 처럼 아래 코드에 numpy randint() 메서드를 활용해 각 컬럼별 데이터 생성한 후 사전을 활용한 DataFrame 객체를 생성하여 출력

```
import numpy as np
import pandas as pd
```

```
def DataFrame_ndarray_dict():
```

```
    StockDate = pd.date_range('2020.09.08', periods=5)
```

→ date index 활용

```
    # 코드 추가
```

```
    print(MyDfObj)
```

```
DataFrame_ndarray_dict()
```

결과물

	ClosePrice	OpenPrice	HighPrice	LowPrice
2020-09-08	5036	5005	5182	4906
2020-09-09	5009	5052	5194	4903
2020-09-10	5041	5021	5157	4970
2020-09-11	5008	5031	5116	4948
2020-09-12	5008	5002	5131	4965

2. 색인(Index) 객체

Pandas index 객체 – 표 형식의 데이터에서 각 행과 열에 대한 이름 저장 객체

- 메타데이터(속성 정보:특정 목적을 갖는 데이터)을 저장하는 객체
- 불변의 배열, 정렬된 집합
- Series, DataFrame 객체 생성시 사용되는 배열이나 순차적인 이름은 내부적으로 Indexing
- 객체의 데이터를 참조하고 수정하게 해주는 역할

2.1 Index 객체

```
import numpy as np
import pandas as pd
```

```
myobj = pd.Series(range(5), index = list('ABCDE'))
```

```
print(myobj.index)
```

```
print(myobj.index[2:])
```

(색인 객체 slicing 연산)

```
myobj.index[0] = 'K' # TypeError
```

Index(['A', 'B', 'C', 'D', 'E'], dtype='object')

Index(['C', 'D', 'E'], dtype='object')

색인 객체는 변경이 불가능

2.2 파이썬 집합과 달리 **행 Index 값**은 중복 허용

```
import numpy as np
import pandas as pd
```

```
myobj2 = pd.Series([3,5,6,9,2], index = ['A','A','B','B','C'])
print(myobj2.index)
```

```
print(myobj2['B'])
```

(중복된 인덱스 선택 시 해당 값의
모든 항목 선택)

Index(['A', 'A', 'B', 'B', 'C'], dtype='object')

```
B    6
B    9
dtype: int64
```

```
dict_data = { 'customer' : ['kim','park','Lee'],
               'pay'       : [5000,6000,7000],
               'local'     : ['seoul','puan','jeju'],
               'card'      : ['9440','4805','4602'] }
```

```
df = pd.DataFrame(dict_data,index=['a','a','b'])
print(df.index)
```

Index(['a', 'a', 'b'], dtype='object')

2.3 재색인(reindex)

Pandas reindex() 메서드

: 새로운 색인에 맞도록 객체를 새로 생성 데이터를 새로운 색인에 맞춰 재배포하고
존재하지 않는 색인값은 NaN 추가

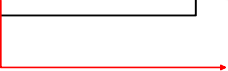
```
import numpy as np
import pandas as pd

seriesobj = pd.Series([3.5, 6.2, 7.8, 9.3] , index=['blue','red','pink','green'])
print(seriesobj)

reindexobj = seriesobj.reindex(['red','green','blue','white'])
print(reindexobj)
```



blue	3.5
red	6.2
pink	7.8
green	9.3
dtype: float64	



red	6.2
green	9.3
blue	3.5
white	NaN
dtype: float64	

reindex method 옵션: ffill -> 직전 값으로 채움 , bfill -> 다음 값으로 채움

fill_value -> 재색인 과정 중 새롭게 나타나는 비워있는 값을 해당 값으로 채움

```
obj1 = pd.Series([2.5, 6.8, 3.2, 9.8], index=[0,2,4,6])  
print(obj1)
```

0	2.5
2	6.8
4	3.2
6	9.8

dtype: float64

```
obj2 = obj1.reindex(range(7), method='ffill')  
print(obj2)
```

0	2.5
1	2.5
2	6.8
3	6.8
4	3.2
5	3.2
6	9.8

dtype: float64

```
obj3 = obj1.reindex(range(7), fill_value = 1.0)  
print(obj3)
```

0	2.5
1	1.0
2	6.8
3	1.0
4	3.2
5	1.0
6	9.8

dtype: float64

DataFrame reindex() 메서드

: DataFrame 로우(색인), 컬럼 reindex 가능 (인덱스, 컬럼 순서 변경에 용이)

```
mydf = pd.DataFrame(np.arange(9).reshape((3,3)), index=['id2','id1','id3'],  
                    columns=['class1','class2','class3'])
```

```
print(mydf)
```

	class1	class2	class3
id2	0	1	2
id1	3	4	5
id3	6	7	8

```
mydf1 = mydf.reindex(['id1','id2','id3','id4'])
```

```
print(mydf1)
```

(Series와 동일하게 인덱스별 로우가 재색인
되며 존재하지 않는 인덱스는 NaN 처리)

	class1	class2	class3
id1	3.0	4.0	5.0
id2	0.0	1.0	2.0
id3	6.0	7.0	8.0
id4	NaN	NaN	NaN

컬럼 재색인은 columns 예약어를 사용해서 재색인

```
classname = ['class1','class5','class2']
```

```
mydf2 = mydf.reindex(columns=classname)
```

```
print(mydf2)
```

	class1	class5	class2
id2	0	NaN	1
id1	3	NaN	4
id3	6	NaN	7

2.4 색인(Index) 객체 실습 문제

실습문제) 생성된 DataFrame 객체에 `reindex` 메서드를 활용하여 다음 결과물과 같은 객체를 생성하도록 코드 구현하시요.

```
import numpy as np
import pandas as pd
```

```
grade_dic = {
    '국어':[92,85,69,75,89],
    '영어':[77,90,95,88,67],
    '수학':[83,73,93,65,73]
}
```

```
mydf = pd.DataFrame(grade_dic,index=['Kim','Lee','Park','Hong','Jung'])
print(mydf)
```

코드 추가

```
printf(reindexobj)
```

	국어	영어	수학
Kim	92	77	83
Lee	85	90	73
Park	69	95	93
Hong	75	88	65
Jung	89	67	73

	국어	영어	수학
Hong	75	88	65
Kim	92	77	83
Lee	85	90	73
Park	69	95	93
Jung	89	67	73
Avg	0	0	0

3. 결측치(NaN) 검색, 제거

3.1 데이터 생성

```
import numpy as np
import pandas as pd
```

```
mydf = pd.DataFrame(np.random.rand(6,4), columns = list('ABCD'))
```

```
print(mydf)
```

```
# 6일치 Datetime64 데이터형 인덱스 생성
```

```
mydf.index = pd.date_range('20170701', periods=6)
```

```
print(mydf.index)
```

```
print(mydf)
```

	A	B	C	D
0	0.738906	0.915665	0.765207	0.223884
1	0.032187	0.030016	0.640464	0.886063
2	0.002580	0.507113	0.001205	0.072578
3	0.138743	0.282877	0.484360	0.103879
4	0.026791	0.694897	0.950129	0.460514
5	0.618647	0.499988	0.426245	0.358837

```
DatetimeIndex(['2017-07-01', '2017-07-02',  
               '2017-07-03', '2017-07-04',  
               '2017-07-05', '2017-07-06'],  
              dtype='datetime64[ns]', freq='D')
```

	A	B	C	D
2017-07-01	0.738906	0.915665	0.765207	0.223884
2017-07-02	0.032187	0.030016	0.640464	0.886063
2017-07-03	0.002580	0.507113	0.001205	0.072578
2017-07-04	0.138743	0.282877	0.484360	0.103879
2017-07-05	0.026791	0.694897	0.950129	0.460514
2017-07-06	0.618647	0.499988	0.426245	0.358837

3.2 isnull() 메서드 : 누락되거나 NaN인 값을 알려주는 불리언값이 저장된 객체를 반환

 notnull() : isnull()과 반대

'F' 컬럼 추가

```
mydf['F'] = [ 3.5, np.NaN, 2.7, 8.2, np.NaN, 7.5 ]
```

NaN이 포함되어 있는 데이터에 대한 Boolean 마스크 생성

NaN인 성분만 True

```
print(mydf.isnull())
```

	A	B	C	D	F
2017-07-01	False	False	False	False	False
2017-07-02	False	False	False	False	True
2017-07-03	False	False	False	False	False
2017-07-04	False	False	False	False	False
2017-07-05	False	False	False	False	True
2017-07-06	False	False	False	False	False

1. 방법 loc인덱서의 행 index 자리에 넣어 NaN을 포함한 행만 추출

isnull() 전체마스크 생성 -> 전체 마스크 중 df["기준열"] --> loc 행 index에 넣기

```
print( mydf.loc[ mydf.isnull()['F'], :] )
```

	A	B	C	D	F
2017-07-02	0.032187	0.030016	0.640464	0.886063	NaN
2017-07-05	0.026791	0.694897	0.950129	0.460514	NaN

2. 방법 : 해당 컬럼기준 NaN Boolean 배열로 로우 선택

```
print( mydf [ mydf['F'].isnull() ] )
```

	A	B	C	D	F
2017-07-02	0.032187	0.030016	0.640464	0.886063	NaN
2017-07-05	0.026791	0.694897	0.950129	0.460514	NaN

```
mydf.iloc[0,0] = np.NaN
```

```
mydf.iloc[3,2] = np.NaN
```

```
print(mydf)
```

컬럼 기준으로 결측 데이터가 하나라도 포함된 행 추출(찾아냄)

```
print( mydf[ mydf.isnull().any(axis=1) ] )
```

	A	B	C	D	F
2017-07-01	NaN	0.780860	0.252845	0.972233	3.5
2017-07-02	0.179948	0.856478	0.948918	0.649527	NaN
2017-07-04	0.029504	0.720839	NaN	0.204720	8.2
2017-07-05	0.141493	0.666752	0.324196	0.150996	NaN

3.3 특정 행,열 데이터 제거 _ drop() : 인덱스 활용 특정 행,열 삭제

```
# 삭제할 NaN 행의 인덱스를 찾았으므로 drop()함수로 행 삭제
# '20170701' 날짜 데이터는 pd.to_datetime()함수로 datetime 데이터형으로 변경
# 변경된 datetime 데이터로 삭제할 index 지정
# 한 행만 삭제 시
print( mydf.drop(pd.to_datetime('20170702')) )
# 여러 행 삭제 시
print( mydf.drop( [pd.to_datetime('20170702'), pd.to_datetime('20170705')] ) )

# inplace = True 옵션으로 원본 데이터 반영
mydf.drop( [ pd.to_datetime('20170702'), pd.to_datetime('20170705') ] , inplace=True )
print(mydf)
```



	A	B	C	D	F
2017-07-01	0.738906	0.915665	0.765207	0.223884	3.5
2017-07-03	0.002580	0.507113	0.001205	0.072578	2.7
2017-07-04	0.138743	0.282877	0.484360	0.103879	8.2
2017-07-05	0.026791	0.694897	0.950129	0.460514	NaN
2017-07-06	0.618647	0.499988	0.426245	0.358837	7.5

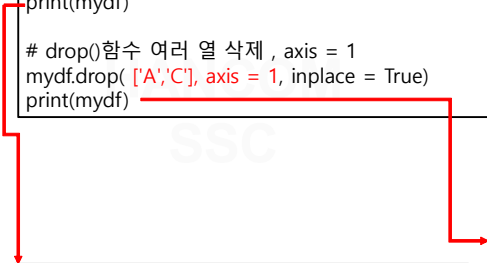


	A	B	C	D	F
2017-07-01	0.738906	0.915665	0.765207	0.223884	3.5
2017-07-03	0.002580	0.507113	0.001205	0.072578	2.7
2017-07-04	0.138743	0.282877	0.484360	0.103879	8.2
2017-07-06	0.618647	0.499988	0.426245	0.358837	7.5

3.4 특정 행,열 데이터 제거 _ drop() : axis = 1 열 삭제

```
# drop()함수 한 열 삭제 , axis = 1  
mydf.drop('F', axis = 1, inplace = True)  
print(mydf)
```

```
# drop()함수 여러 열 삭제 , axis = 1  
mydf.drop(['A','C'], axis = 1, inplace = True)  
print(mydf)
```



	A	B	C	D
2017-07-01	0.738906	0.915665	0.765207	0.223884
2017-07-03	0.002580	0.507113	0.001205	0.072578
2017-07-04	0.138743	0.282877	0.484360	0.103879
2017-07-06	0.618647	0.499988	0.426245	0.358837

	B	D
2017-07-01	0.915665	0.223884
2017-07-03	0.507113	0.072578
2017-07-04	0.282877	0.103879
2017-07-06	0.499988	0.358837

3.5 특정 열 데이터 제거 _ del 키워드 : 컬럼 지정으로 해당 열 삭제

```
import numpy as np
import pandas as pd
mydf = pd.DataFrame(np.random.rand(6,4), columns = list('ABCD'))
print(mydf)
mydf.index = pd.date_range('20170701',periods=6) # 6일치 Datetime64 데이터형 인덱스 생성
print(mydf.index)
print(mydf)
# 'F' 컬럼 추가
mydf['F'] = [ 3.5, np.NaN, 2.7, 8.2, np.NaN, 7.5 ]
print(mydf)

# del 키워드 1개 열 삭제
del mydf['F']
print(mydf)

# del 키워드 여러 열 삭제
columns = [ 'A' , 'C' ]
for col in columns:
    del mydf[col]
print(mydf)
```

3.6 결측 데이터 행 제거 _ dropna() : 결측 데이터 존재하는 행 삭제

결측 데이터 포함한 데이터프레임 생성

```
import numpy as np
import pandas as pd
```

```
mydf = pd.DataFrame(np.random.rand(6,4), columns = list('ABCD'))
```

```
print(mydf)
```

```
mydf.index = pd.date_range('20170701', periods=6) # 6일치 Datetime64 데이터형 인덱스 생성
```

```
print(mydf.index)
```

```
print(mydf)
```

```
# 'F' 컬럼 추가
```

```
mydf['F'] = [ 3.5, np.NaN, 2.7, 8.2, np.NaN, 7.5 ]
```

mydf.dropna(how = 'any') : 각 행들 중 NaN이 하나라도 있으면 그 행을 제거

```
print( mydf.dropna( how = 'any') )
```



	A	B	C	D	F
2017-07-01	0.227859	0.667275	0.731589	0.411623	3.5
2017-07-03	0.437222	0.311739	0.795300	0.931547	2.7
2017-07-04	0.135811	0.652932	0.122618	0.659533	8.2
2017-07-06	0.529992	0.178933	0.908414	0.609782	7.5



특정 컬럼(열) 기준 NaN 데이터 행 삭제

```
mydf.dropna(subset=['F'], axis=0, inplace=True)
print(mydf)
```

```
# loc 인덱서 활용 새로운 행 추가 하고 NaN 데이터로 채움  
mydf.loc[pd.to_datetime('20180203')] = np.NaN  
print(mydf)
```

```
# mydf.dropna(how = 'all') : 각 행의 성분이 모두 NaN 일때 해당 행 제거  
print(mydf.dropna(how = 'all'))
```

	A	B	C	D	F
2017-07-01	0.227859	0.667275	0.731589	0.411623	3.5
2017-07-02	0.666653	0.460818	0.485995	0.116607	NaN
2017-07-03	0.437222	0.311739	0.795300	0.931547	2.7
2017-07-04	0.135811	0.652932	0.122618	0.659533	8.2
2017-07-05	0.483350	0.581480	0.774917	0.715936	NaN
2017-07-06	0.529992	0.178933	0.908414	0.609782	7.5
2018-02-03	NaN	NaN	NaN	NaN	NaN

	A	B	C	D	F
2017-07-01	0.227859	0.667275	0.731589	0.411623	3.5
2017-07-02	0.666653	0.460818	0.485995	0.116607	NaN
2017-07-03	0.437222	0.311739	0.795300	0.931547	2.7
2017-07-04	0.135811	0.652932	0.122618	0.659533	8.2
2017-07-05	0.483350	0.581480	0.774917	0.715936	NaN
2017-07-06	0.529992	0.178933	0.908414	0.609782	7.5

3.7 결측 데이터 설정 _ fillna() : 결측 데이터 일괄 특정 value로 설정

```
# mydf.fillna(value = ) : 결측치에 지정된 value값 일괄 대입, 원본 변경 : inplace = True
print( mydf.fillna(value = 3.8) )

print(mydf)
#inplace = True 옵션으로 원본 데이터 변경
mydf.dropna(how = 'any', inplace = True)
print(mydf)
```

	A	B	C	D	F
2017-07-01	0.227859	0.667275	0.731589	0.411623	3.5
2017-07-02	0.666653	0.460818	0.485995	0.116607	3.8
2017-07-03	0.437222	0.311739	0.795300	0.931547	2.7
2017-07-04	0.135811	0.652932	0.122618	0.659533	8.2
2017-07-05	0.483350	0.581480	0.774917	0.715936	3.8
2017-07-06	0.529992	0.178933	0.908414	0.609782	7.5
2018-02-03	3.800000	3.800000	3.800000	3.800000	3.8

	A	B	C	D	F
2017-07-01	0.227859	0.667275	0.731589	0.411623	3.5
2017-07-03	0.437222	0.311739	0.795300	0.931547	2.7
2017-07-04	0.135811	0.652932	0.122618	0.659533	8.2
2017-07-06	0.529992	0.178933	0.908414	0.609782	7.5

3.8 row, column 데이터 추가 및 변경

```
import numpy as np
import pandas as pd

people_seoul = pd.read_excel('population_in_seoul.xls')

print(people_seoul)
people_seoul.drop(0,inplace=True)

# Column 추가
people_seoul['도시'] = '서울'
print(people_seoul)

# 특정 Column 데이터 변경
people_seoul.loc[ people_seoul['자치구'] == '금천구' , '도시' ] = '경기'
print(people_seoul)

# row(행) 데이터 추가 , ignore_index=True 해줘야함
people_seoul = people_seoul.append({'자치구':'임시구','남자':2000,'여자':3000,'고령자':4000,'도시':'서울'},ignore_index=True)
print(people_seoul)
```

3.9 결측치(NaN) 검색, 제거 실습 문제

실습문제) 다음 엑셀 데이터를 이용해 생성된 DataFrame 객체에 아래 결과물과 같은 결측 데이터 제거 객체를 생성하도록 코드 구현하시요.

```
import numpy as np
import pandas as pd
```

```
people_seoul = pd.read_excel('population_in_seoul.xls')
print(people_seoul)
```

```
columns = ['남자', '여자', '고령자']
```

```
# 반복문 활용 각 컬럼별 결측(NaN)데이터를 포함하는 행 데이터 출력
```

```
# 코드 추가
```

	자치구	남자	여자	고령자
0	합계	4751696.0	4988702.0	1468146.0
1	종로구	73952.0	77815.0	27394.0
2	중구	NaN	64248.0	23025.0
3	용산구	110478.0	118352.0	38531.0
4	성동구	148510.0	154648.0	43662.0
5	광진구	171103.0	181589.0	47347.0
6	동대문구	172440.0	NaN	58764.0
7	종랑구	197927.0	200885.0	64449.0
8	성북구	NaN	227496.0	70204.0
9	강북구	153356.0	160734.0	59808.0
10	도봉구	163953.0	171327.0	58070.0
11	구로구	201077.0	205671.0	NaN
12	금천구	118471.0	114900.0	37191.0
13	영등포구	182913.0	185489.0	57350.0
14	동작구	192829.0	204789.0	61607.0
15	관악구	251923.0	NaN	74518.0
16	서초구	206185.0	224842.0	56455.0
17	강남구	260305.0	283723.0	70029.0
18	송파구	328073.0	349416.0	86062.0
19	강동구	210593.0	215626.0	NaN

xlrd 모듈 설치
: 1.2.0 version

```
자치구  남자   여자   고령자
2  중구  NaN   64248.0  23025.0
8  성북구  NaN   227496.0  70204.0
=====
자치구  남자  여자   고령자
6  동대문구  172440.0  NaN   58764.0
15  관악구  251923.0  NaN   74518.0
=====
자치구  남자   여자   고령자
11  구로구  201077.0  205671.0  NaN
19  강동구  210593.0  215626.0  NaN
=====
```

people_seoul 원본 객체에 NaN(결측)데이터가 하나라도 있는 행
데이터 삭제 후 반영

코드 추가

첫번째 합계 행 데이터 삭제 후 원본 데이터 반영

코드 추가

고령자 컬럼 데이터 삭제

인덱스 재 정리

코드 추가

	자치구	남자	여자	고령자
0	합계	4751696.0	4988702.0	1468146.0
1	종로구	73952.0	77815.0	27394.0
3	용산구	110478.0	118352.0	38531.0
4	성동구	148510.0	154648.0	43662.0
5	광진구	171103.0	181589.0	47347.0
7	중랑구	197927.0	200885.0	64449.0
9	강북구	153356.0	160734.0	59808.0
10	도봉구	163953.0	171327.0	58070.0
12	금천구	118471.0	114900.0	37191.0
13	영등포구	182913.0	185489.0	57350.0
14	동작구	192829.0	204789.0	61607.0
16	서초구	206185.0	224842.0	56455.0
17	강남구	260305.0	283723.0	70029.0
18	송파구	328073.0	349416.0	86062.0

	자치구	남자	여자	고령자
1	종로구	73952.0	77815.0	27394.0
3	용산구	110478.0	118352.0	38531.0
4	성동구	148510.0	154648.0	43662.0
5	광진구	171103.0	181589.0	47347.0
7	중랑구	197927.0	200885.0	64449.0
9	강북구	153356.0	160734.0	59808.0
10	도봉구	163953.0	171327.0	58070.0
12	금천구	118471.0	114900.0	37191.0
13	영등포구	182913.0	185489.0	57350.0
14	동작구	192829.0	204789.0	61607.0
16	서초구	206185.0	224842.0	56455.0
17	강남구	260305.0	283723.0	70029.0
18	송파구	328073.0	349416.0	86062.0

	자치구	남자	여자
0	종로구	73952.0	77815.0
1	용산구	110478.0	118352.0
2	성동구	148510.0	154648.0
3	광진구	171103.0	181589.0
4	중랑구	197927.0	200885.0
5	강북구	153356.0	160734.0
6	도봉구	163953.0	171327.0
7	금천구	118471.0	114900.0
8	영등포구	182913.0	185489.0
9	동작구	192829.0	204789.0
10	서초구	206185.0	224842.0
11	강남구	260305.0	283723.0
12	송파구	328073.0	349416.0

4. 데이터 선택 (단일색인, 슬라이싱, 불린색인, loc , iloc)

4.1 Series 활용 데이터 생성 후 특정 컬럼 선택

```
import numpy as np
import pandas as pd
```

```
sobj1 = pd.Series([3.5,6.9,2.8,np.NaN,7.5],index=list('ABCDE'))
sobj2 = pd.Series([9.2,np.NaN,7.7,2.5,8.2],index=list('ABCDE'))
sobj3 = pd.Series([1.5,2.6,3.7,4.8,5.9],index=list('ABCDE'))
print(sobj1)
```

리스트의 각 항목이 DataFrame의 로우가 됨

Series의 색인이 DataFrame의 컬럼이 됨

```
mydf = pd.DataFrame([sobj1,sobj2,sobj3])
print(mydf)
```

('B' 컬럼 데이터 선택)

```
print(mydf['B'])
print(type(mydf['B']))
```

(여러 개 특정 컬럼 데이터 선택)

```
print(mydf[ ['A','C','D'] ] )
```

	A	B	C	D	E
0	3.5	6.9	2.8	NaN	7.5
1	9.2	NaN	7.7	2.5	8.2
2	1.5	2.6	3.7	4.8	5.9

0	6.9
1	NaN
2	2.6

Name: B, dtype: float64

<class 'pandas.core.series.Series'>

	A	C	D
0	3.5	2.8	NaN
1	9.2	7.7	2.5
2	1.5	3.7	4.8

4.2 데이터 선택 (Dataframe 특정 열 선택)

```
import numpy as np
import pandas as pd
sobj1 = pd.Series([3.5,6.9,2.8,np.NaN,7.5],index=list('ABCDE'))
sobj2 = pd.Series([9.2,np.NaN,7.7,2.5,8.2],index=list('ABCDE'))
sobj3 = pd.Series([1.5,2.6,3.7,4.8,5.9],index=list('ABCDE'))
mydf = pd.DataFrame([sobj1,sobj2,sobj3])
print(mydf)
```

	A	B	C	D	E
0	3.5	6.9	2.8	NaN	7.5
1	9.2	NaN	7.7	2.5	8.2
2	1.5	2.6	3.7	4.8	5.9

```
mydf1 = mydf.rename(index = {0:'one',1:'two',2:'three'})
print(mydf1)
```



(index 이름 변경)

	A	B	C	D	E
one	3.5	6.9	2.8	NaN	7.5
two	9.2	NaN	7.7	2.5	8.2
three	1.5	2.6	3.7	4.8	5.9

단일열 Or 여러열 선택

```
print(mydf1[ ['A'] ])
```

(DataFrame 형성)

	A
one	3.5
two	9.2
three	1.5

```
print(mydf1[ ['A','C','E'] ])
```

(mydf1[] 대괄호 안에
컬럼 값을 입력하여 단일 열 선택)

	A	C	E
one	3.5	2.8	7.5
two	9.2	7.7	8.2
three	1.5	3.7	5.9

4.3 데이터 선택(슬라이싱 활용 특정 행 선택)

```
mydf1 = mydf.rename(index = {0:'one',1:'two',2:'three'})  
print(mydf1)
```

#슬라이싱을 이용한 로우 선택

```
print(mydf1[0:2])
```

(인덱싱을 이용한 슬라이싱 선택의
경우 마지막 인덱스 포함하지 않음)

```
print(mydf1['two':'three'])
```

	A	B	C	D	E
one	3.5	6.9	2.8	NaN	7.5
two	9.2	NaN	7.7	2.5	8.2
three	1.5	2.6	3.7	4.8	5.9

	A	B	C	D	E
one	3.5	6.9	2.8	NaN	7.5
two	9.2	NaN	7.7	2.5	8.2

(라벨 이름을 이용한 슬라이싱 선택
의 경우 시작과 끝점을 포함)

	A	B	C	D	E
two	9.2	NaN	7.7	2.5	8.2
three	1.5	2.6	3.7	4.8	5.9

선택된 영역에 값 대입

```
print(mydf1)
```

'A'컬럼의 0~1행 선택

```
mydf1['A'][0:2] = 9.8
```

```
print(mydf1)
```

	A	B	C	D	E
one	3.5	6.9	2.8	NaN	7.5
two	9.2	NaN	7.7	2.5	8.2
three	1.5	2.6	3.7	4.8	5.9

	A	B	C	D	E
one	9.8	6.9	2.8	NaN	7.5
two	9.8	NaN	7.7	2.5	8.2
three	1.5	2.6	3.7	4.8	5.9

Series 객체

**** 주의**



데이터프레임 객체 경우
mydf1[0] # 암묵적 인덱싱으로 접근 오류
mydf1[0:1] # 슬라이싱으로 접근해야 함
Series 객체의 경우
mydf1['A'][0] # 접근 가능

4.4 데이터 선택(불리언 배열 활용 특정 영역 선택)

```
import numpy as np
import pandas as pd

mydf = pd.DataFrame(np.random.randint(10,size=(4,4)),
                    index=['one','two','three','four'],columns=list('ABCD'))
print(mydf)
```

```
print(mydf < 5)
```

(스칼라 비교로 생성된 불리언 DataFrame 객체)

```
print(mydf[ mydf['B'] < 5 ])
```

(‘B’컬럼 기준 불리언 배열을 이용한 특정 영역 선택)

```
mydf[ mydf < 5 ] = 0
```

```
print(mydf)
```

(불리언 배열의 True인 위치에 0을 대입)

	A	B	C	D
one	5	3	8	5
two	9	6	8	6
three	2	7	3	2
four	4	0	5	7

	A	B	C	D
one	False	True	False	False
two	False	False	False	False
three	True	False	True	True
four	True	True	False	False

	A	B	C	D
one	5	3	8	5
four	4	0	5	7

	A	B	C	D
one	5	0	8	5
two	9	6	8	6
three	0	7	0	0
four	0	0	5	7

4.5 데이터 선택(loc 인덱서 활용 : 라벨로 로우, 컬럼 선택)

loc : label-location

- 데이터프레임의 행이나 컬럼에 **label** 이나 **boolean** 배열로 접근

```
import numpy as np
import pandas as pd

mydf = pd.DataFrame(np.arange(16).reshape(4,4),
                    index=['one','two','three','four'],columns=list('ABCD'))
print(mydf)
```

```
print(mydf[0:2])
```

(슬라이싱 시 암묵적 인덱스 활용 특
정 행 선택)

```
print(mydf.loc['one':'three'])
```

(시작지점 ~ 종료지점 포함 라벨
활용한 로우 선택)

```
print( mydf.loc[ : , ['A','C'] ] )
```

mydf.loc [시작지점:종료지점, ['컬럼명', '컬럼명',]]
활용한 두 컬럼의 모든 로우 데이터 선택

* loc[:] 경우, Index가 숫자형으로 되어있을 때 숫자형으로 데이터로 인자
입력, 하지만 슬라이싱과 달리 명칭 기반으로 해석하기 때문에
[시작 값 : 종료 값]은 시작값~종료값까지를 의미

	A	B	C	D
one	0	1	2	3
two	4	5	6	7
three	8	9	10	11
four	12	13	14	15

	A	B	C	D
one	0	1	2	3
two	4	5	6	7

	A	B	C	D
one	0	1	2	3
two	4	5	6	7
three	8	9	10	11

	A	C
one	0	2
two	4	6
three	8	10
four	12	14

```
import numpy as np
import pandas as pd
```

```
mydf = pd.DataFrame(np.arange(16).reshape(4,4),
                    index=['one','two','three','four'],columns=list('ABCD'))
```

```
print(mydf)
```

```
print( mydf.loc[ : 'three' , 'C' ] )
```

('C' 컬럼 기준 처음부터 'three' 라벨
까지 로우 데이터 출력)

다음은 위 loc 활용 데이터 선택과 동일한 결과 코드

```
print( mydf['C'][:3] ) # 'C' 컬럼 Series에 0~2 인덱스 데이터 선택
```

```
print( mydf.loc[ 'one':'two', ['A','B'] ] )
```

(loc 활용 데이터프레임 객체 일부분 선택)

Boolean 인덱싱 활용 데이터 선택

```
print(mydf)
```

```
print( mydf[ mydf['C']<10 ] ) # 'C' 컬럼 기준 10보다 작은 DataFrame Data 선택
```

```
print(mydf.loc[ mydf['C'] < 10 , : ]) # 조건에 해당하는 전체 Column Data
```

```
print(mydf.loc[ mydf['C'] < 10 , ['A','D']]) # 조건에 해당하는 'A', 'D' Column Data
```

```
print(mydf.loc[ mydf['C'] < 10, 'A':'C']) # 조건에 해당하는 'A'~'C' Column Data
```

	A	B	C	D
one	0	1	2	3
two	4	5	6	7
three	8	9	10	11
four	12	13	14	15

one	2
two	6
three	10

one	2
two	6
three	10

	A	B
one	0	1
two	4	5

★ 특정컬럼 선택시 loc 활용

4.6 loc 인덱서 활용 데이터 변경 및 추가

```
import numpy as np
import pandas as pd

mydf = pd.DataFrame(np.arange(16).reshape(4,4),
                    index=['one','two','three','four'],columns=list('ABCD'))
print(mydf)
```

	A	B	C	D
one	0	1	2	3
two	4	5	6	7
three	8	9	10	11
four	12	13	14	15

```
mydf.loc['four','A'] = 30
print(mydf)
```

(특정 위치 선택 후 하나의 값 변경)

	A	B	C	D
one	0	1	2	3
two	4	5	6	7
three	8	9	10	11
four	30	13	14	15

```
mydf.loc[:, 'C'] = 50
print(mydf)
```

(특정 컬럼 선택 후 전체 데이터 변경)

	A	B	C	D
one	0	1	50	3
two	4	5	50	7
three	8	9	50	11
four	30	13	50	15

```
# 새로운 행이 필요할 때 사용하면 용이
mydf.loc['five'] = 0
print(mydf)
```

(새로운 로우 인덱스 추가 후 데이터 추가)

	A	B	C	D
one	0	1	50	3
two	4	5	50	7
three	8	9	50	11
four	30	13	50	15
five	0	0	0	0

4.7 iloc 인덱서 활용 : 정수색인을 통한 로우, 컬럼 선택

```
import numpy as np
import pandas as pd

mydf = pd.DataFrame(np.arange(16).reshape(4,4),
                    index=['one','two','three','four'],columns=list('ABCD'))
print(mydf)
```

정수 2 index 행 선택

```
print(mydf.iloc[2])
```

슬라이싱을 이용한 특정 영역 선택

```
print(mydf.iloc[2:4,0:2])
```

mydf.iloc [시작지점:종료지점, 시작지점:종료지점]
: list 슬라이싱과 동일하게 종료지점 index는 포함하지 않음

특정 값 선택

```
print(mydf.iloc[1,1])
```

- 명칭 인덱스 입력 오류
- 슬라이싱, 팬시 인덱싱은 제공하나
불린 인덱싱은 제공하지 않음!!

iloc : interger-location

- 위치 기반 인덱싱만 허용,
- 대괄호([]) 안에 들어가는 요소는 모두 **integer, inter**형의 슬라이싱, 팬시 리스트 값을 입력

0부터 시작하는 각 행의 정수 index

		A	B	C	D
0	one	0	1	2	3
1	two	4	5	6	7
2	three	8	9	10	11
3	four	12	13	14	15

A	8
B	9
C	10
D	11

Name: three, dtype: int32

A	B
three	8 9
four	12 13

```
import numpy as np
import pandas as pd
```

```
mydf = pd.DataFrame(np.arange(16).reshape(4,4),
                    index=['one','two','three','four'],columns=list('ABCD'))
print(mydf)
```

범위가 아닌 list 활용 특정 영역 선택

```
print(mydf.iloc[ [0,2],[1,3] ]) ( 0, 2 index 행의 1, 3 index 열 선택)
```

열 부분의 시작지점과 종료지점이 없는 경우

```
print( mydf.iloc[1: , : ] ) ( 모든 열 선택)
```

행 부분의 시작지점과 종료지점이 없는 경우

```
print( mydf.iloc[ : , :2 ] ) ( 모든 행 선택)
```

column idx

행 idx	0	1	2	3
0	one	0	1	2
1	two	4	5	6
2	three	8	9	10
3	four	12	13	14

	B	D
one	1	3
three	9	11

	A	B	C	D
two	4	5	6	7
three	8	9	10	11
four	12	13	14	15

	A	B
one	0	1
two	4	5
three	8	9
four	12	13

4.8 선택 객체 수정 Warning

```
import numpy as np
import pandas as pd
```

: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame
(데이터프레임 슬라이스 복사본에 값을 설정하려고 하고 있다는 경고)

```
mydf = pd.DataFrame(np.arange(16).reshape(4,4),
                    index=['one','two','three','four'],columns=list('ABCD'))
print(mydf)
```

**Numpy 배열로 형성된
Dataframe 객체의 문제**

```
mydf2 = mydf[ mydf['D'] > 10 ] # 불린색인 선택
print(mydf2)
mydf2.loc['four','D'] = 80
print(mydf)
print(mydf2)
```

Warning : mydf2 = mydf[mydf['D'] > 10].copy() 활용

```
mydf3 = mydf['one':'three'].copy() # 슬라이싱 선택
print(mydf3)
mydf3.loc['two','B'] = 30
print(mydf)
print(mydf3)
```

**# numpy 객체와 달리 numpy 배열로 형성된 DataFrame 객체의 경우
그 DataFrame 객체에 일부분만 복사해온건데, 이것을 수정할 경우 복사본만 수정할지
원본도 수정할지 알 수가 없어서 나오는 경고**

불린, 슬라이싱으로 추출한 객체의 복사본을 이용하여 복사본 객체에 데이터를 write 하면
완벽히 원본 객체와 구분할 수 있음
따라서 추출한 객체의 copy() 활용하여 복사본 객체 생성하여 활용

4.9 데이터 선택 실습 문제

실습문제) 다음 `population_in_seoul.xls` 데이터를 활용하여 남자 인구수가 20만 이상인 서울시 자치구 남자 인구수 데이터를 표현 하시오.

```
import numpy as np
import pandas as pd
```

```
people_seoul = pd.read_excel('population_in_seoul.xls')
```

```
print(" drop 메서드 활용 첫번째 합계 행 삭제 ? ")
people_seoul.drop(0,inplace=True)
```

```
print(" fillna 메서드 활용 : 결측 데이터를 0 으로 설정 ? ")
# 코드 추가
```

```
print("남자 인구 20만 이상인 Boolean index 색인을 이용한 DataFrame 선택 ? ")
# 코드 추가
```

```
print("loc 활용 남자 인구 20만 이상인 자치구의 남자 인구 데이터 출력 ? ")
# 코드 추가
print(SelData)
```

```
print("reset_index 메서드 활용 index 초기화 ?")
# 코드 추가
print(Reindex_SelData)
```

```
# 자치구가 '강남구','관악구' 인 두 행 데이터 추출 ?
GuData = ['강남구','관악구']
# isin() 메서드 활용
```

자치구	남자	여자	고령자
11 구로구	201077.0	205671.0	0.0
15 관악구	251923.0	0.0	74518.0
16 서초구	206185.0	224842.0	56455.0
17 강남구	260305.0	283723.0	70029.0
18 송파구	328073.0	349416.0	86062.0
19 강동구	210593.0	215626.0	0.0

자치구	남자
11 구로구	201077.0
15 관악구	251923.0
16 서초구	206185.0
17 강남구	260305.0
18 송파구	328073.0
19 강동구	210593.0

자치구	남자
0 구로구	201077.0
1 관악구	251923.0
2 서초구	206185.0
3 강남구	260305.0
4 송파구	328073.0
5 강동구	210593.0

자치구	남자	여자	고령자
15 관악구	251923.0	0.0	74518.0
17 강남구	260305.0	283723.0	70029.0

Thank You

